SQL Exercises

Practice Problems with Answers

30 exercises from Easy to Hard - based on your database tables

Your Database Tables

users

id	name
1	Jaime
2	Jon Snow
3	Theon Greyjoy

swords

id	name	attack	owner_id
1	Oathkeeper	200	1
2	Wolf Bane	214	2
3	KrakenSlayer	185	3

Instructions: Write the SQL query that answers each question. Answers are at the end of the PDF.

Section A: Easy (SELECT, WHERE, ORDER BY)

Q1.

Select all columns from the users table.

[Easy]

Q2.

Select only the name column from the users table.

[Easy]

Q3.

Find the user whose name is 'Jon Snow'.

[Easy]

Q4.

Find all swords with an attack value greater than 190.

[Easy]

Q5.

Select all users sorted by name in alphabetical order (A-Z).

[Easy]

Q6.

Select all swords sorted by attack power from highest to lowest.

[Easy]

Q7.

Find swords with attack between 185 and 200 (inclusive).

[Easy]

Q8.

Find users whose name starts with 'J'.

[Easy]

Q9.

Select only the first 2 users from the users table.

[Easy]

Q10.

Find users whose name is either 'Jaime' or 'Theon Greyjoy' using IN.

[Easy]

Section B: Medium (JOIN, Aggregates, GROUP BY)

Q11.

Count the total number of users.

[Medium]

Q12.

Find the sword with the highest attack value. Show only the name and attack.

[Medium]

Q13.

Calculate the average attack power of all swords.

[Medium]

Q14.

Find the total (SUM) of all sword attack values.

[Medium]

Q15.

Join users and swords to show each user's name alongside their sword name.

[Medium]

Q16.

Join users and swords, but only show users whose sword has attack > 200.

[Medium]

Q17.

Show each user's name, sword name, and attack - sorted by attack descending.

[Medium]

Q18.

Insert a new user named 'Arya Stark' into the users table.

[Medium]

Q19.

Update the attack value of 'KrakenSlayer' to 195.

[Medium]

Q20.

Delete the user named 'Arya Stark' from the users table.

[Medium]

Section C: Hard (Subqueries, CASE, Window Functions, CTEs)

Q21.

Find the user who owns the sword with the highest attack value. Use a subquery.

[Hard]

Q22.

Using CASE WHEN, label each sword as 'Legendary' if attack >= 210, 'Epic' if >= 195, or 'Common' otherwise. Show name, attack, and the label.

[Hard]

Q23.

Use a CTE (WITH) to first find swords with attack > 190, then join with users to show the owner's name.

[Hard]

Q24.

Use ROW_NUMBER() to rank all swords by attack power (highest first). Show name, attack, and rank.

[Hard]

Q25.

Find users who own a sword using EXISTS (not JOIN or IN).

[Hard]

Q26.

Combine user names and sword names into a single column called 'all_names' using UNION.

[Hard]

Q27.

Find the owner_id that has the sword with the minimum attack, using a subquery in WHERE.

[Hard]

Q28.

Using LEFT JOIN, find users who do NOT have a sword (assume some users might not). Show only users where the sword is NULL.

[Hard]

Q29.

Write a query that shows each user's name, their sword's attack, and what percentage of the total attack their sword represents. (Hint: use a subquery for total.)

[Hard]

Q30.

Write a transaction that inserts a new user 'Sansa Stark' (id=4) and a new sword 'Needle' (attack=150, owner_id=4), then commits.

[Hard]

ANSWERS

Compare your queries with these solutions. Multiple correct answers may exist!

Section A: Easy

Q1. Select all columns from users

```
SELECT * FROM users;
```

Expected Result:

id	name
1	Jaime
2	Jon Snow
3	Theon Greyjoy

Q2. Select only the name column

```
SELECT name FROM users;
```

Expected Result:

name
Jaime
Jon Snow
Theon Greyjoy

Q3. Find Jon Snow

```
SELECT * FROM users WHERE name = 'Jon Snow';
```

Expected Result:

id	name
2	Jon Snow

Q4. Swords with attack > 190

```
SELECT * FROM swords WHERE attack > 190;
```

Expected Result:

id	name	attack	owner_id
1	Oathkeeper	200	1
2	Wolf Bane	214	2

Q5. Users sorted A-Z

```
SELECT * FROM users ORDER BY name ASC;
```

Expected Result:

id	name
1	Jaime

2	Jon Snow
3	Theon Greyjoy

Q6. Swords sorted by attack DESC

```
SELECT * FROM swords ORDER BY attack DESC;
```

Expected Result:

id	name	attack	owner_id
2	Wolf Bane	214	2
1	Oathkeeper	200	1
3	KrakenSlayer	185	3

Q7. Swords with attack between 185 and 200

```
SELECT * FROM swords WHERE attack BETWEEN 185 AND 200;
```

Expected Result:

id	name	attack	owner_id
1	Oathkeeper	200	1
3	KrakenSlayer	185	3

Q8. Users whose name starts with 'J'

```
SELECT * FROM users WHERE name LIKE 'J%';
```

Expected Result:

id	name
1	Jaime
2	Jon Snow

Q9. First 2 users

```
SELECT * FROM users LIMIT 2;
```

Expected Result:

id	name
1	Jaime
2	Jon Snow

Q10. Jaime or Theon using IN

```
SELECT * FROM users WHERE name IN ('Jaime', 'Theon Greyjoy');
```

Expected Result:

id	name
1	Jaime
3	Theon Greyjoy

Section B: Medium

Q11. Count total users

```
SELECT COUNT(*) FROM users;
```

Expected Result:

count
3

Q12. Sword with highest attack

```
SELECT name, attack FROM swords ORDER BY attack DESC LIMIT 1;
```

Expected Result:

name	attack
Wolf Bane	214

Q13. Average attack

```
SELECT AVG(attack) FROM swords;
```

Expected Result:

avg
199.67

Q14. Sum of all attack values

```
SELECT SUM(attack) FROM swords;
```

Expected Result:

sum
599

Q15. Join users and swords

```
SELECT users.name, swords.name AS sword
FROM users
JOIN swords ON users.id = swords.owner_id;
```

Expected Result:

name	sword
Jaime	Oathkeeper
Jon Snow	Wolf Bane
Theon Greyjoy	KrakenSlayer

Q16. Join with attack > 200

```
SELECT users.name, swords.name AS sword, swords.attack
FROM users
JOIN swords ON users.id = swords.owner_id
WHERE swords.attack > 200;
```

Expected Result:

name	sword	attack
Jon Snow	Wolf Bane	214

Q17. Join sorted by attack DESC

```
SELECT users.name, swords.name AS sword, swords.attack
FROM users
JOIN swords ON users.id = swords.owner_id
ORDER BY swords.attack DESC;
```

Expected Result:

name	sword	attack
Jon Snow	Wolf Bane	214
Jaime	Oathkeeper	200
Theon Greyjoy	KrakenSlayer	185

Q18. Insert Arya Stark

```
INSERT INTO users (name) VALUES ('Arya Stark');
```

After running, users table will have 4 rows with Arya Stark as id=4.

Q19. Update KrakenSlayer attack

```
UPDATE swords SET attack = 195 WHERE name = 'KrakenSlayer';
```

Expected Result:

KrakenSlayer's attack is now 195 instead of 185.

Q20. Delete Arya Stark

```
DELETE FROM users WHERE name = 'Arya Stark';
```

Arya is removed. Table is back to 3 users.

Section C: Hard

Q21. User with highest attack sword (subquery)

```
SELECT * FROM users
WHERE id = (
  SELECT owner_id FROM swords
  ORDER BY attack DESC LIMIT 1
);
```

Expected Result:

id	name
2	Jon Snow

Q22. CASE WHEN for sword rarity

```
SELECT name, attack,
  CASE
    WHEN attack >= 210 THEN 'Legendary'
    WHEN attack >= 195 THEN 'Epic'
    ELSE 'Common'
  END AS rarity
FROM swords;
```

Expected Result:

name	attack	rarity
Oathkeeper	200	Epic
Wolf Bane	214	Legendary
KrakenSlayer	185	Common

Q23. CTE for powerful swords

```
WITH strong_swords AS (
  SELECT owner_id, name AS sword, attack
  FROM swords WHERE attack > 190
)
SELECT u.name, ss.sword, ss.attack
FROM users u
JOIN strong_swords ss ON u.id = ss.owner_id;
```

Expected Result:

name	sword	attack
Jaime	Oathkeeper	200
Jon Snow	Wolf Bane	214

Q24. ROW_NUMBER() to rank swords

```
SELECT name, attack,
  ROW_NUMBER() OVER (ORDER BY attack DESC) AS rank
FROM swords;
```

Expected Result:

name	attack	rank
Wolf Bane	214	1
Oathkeeper	200	2
KrakenSlayer	185	3

Q25. EXISTS to find sword owners

```
SELECT * FROM users u
WHERE EXISTS (
  SELECT 1 FROM swords s
  WHERE s.owner_id = u.id
);
```

Expected Result:

id	name
1	Jaime
2	Jon Snow
3	Theon Greyjoy

Q26. UNION user names and sword names

```
SELECT name AS all_names FROM users
UNION
SELECT name FROM swords;
```

Expected Result:

all_names
Jaime
Jon Snow
Theon Greyjoy
Oathkeeper
Wolf Bane
KrakenSlayer

Q27. Owner of sword with min attack (subquery)

```
SELECT owner_id FROM swords
WHERE attack = (SELECT MIN(attack) FROM swords);
```

Expected Result:

owner_id
3

owner_id 3 = Theon Greyjoy, who owns KrakenSlayer (attack: 185)

Q28. LEFT JOIN to find users without swords

```
SELECT u.id, u.name
FROM users u
LEFT JOIN swords s ON u.id = s.owner_id
WHERE s.id IS NULL;
```

Expected Result:

(empty result - all 3 users currently have swords. If a user without a sword existed, they would appear here.)

Q29. Attack percentage of total

```
SELECT u.name, s.attack,
  ROUND(s.attack * 100.0 / (SELECT SUM(attack) FROM swords), 1)
    AS pct_of_total
FROM users u
JOIN swords s ON u.id = s.owner_id;
```

Expected Result:

name	attack	pct_of_total
Jaime	200	33.4
Jon Snow	214	35.7
Theon Greyjoy	185	30.9

Q30. Transaction: insert user + sword

```
BEGIN;
  INSERT INTO users (id, name)
    VALUES (4, 'Sansa Stark');
  INSERT INTO swords (name, attack, owner_id)
    VALUES ('Needle', 150, 4);
COMMIT;
```

Expected Result:

After COMMIT, both tables have 4 rows. Sansa owns Needle (attack: 150).

id	name
1	Jaime
2	Jon Snow
3	Theon Greyjoy
4	Sansa Stark

id	name	attack	owner_id
1	Oathkeeper	200	1
2	Wolf Bane	214	2
3	KrakenSlayer	185	3
4	Needle	150	4